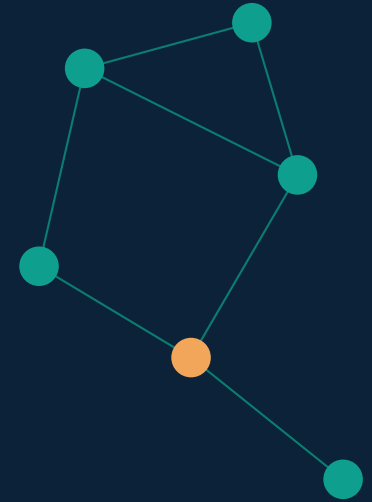


■ BUILDING AGENTIC AI · 2026

에이전틱 AI 구축 가이드

개념·설계 원칙 → 하네스 → 컨텍스트 → 도구·표준 → 멀티 에이전트 → 평가까지, 핵심만 정리

스터디 발표 자료 · Anthropic·OpenAI·Microsoft·Google 공식 자료 기반 (2026-06 검증, 출처는 마지막 장)



CONTENTS

다섯 파트로 정리한 구축 가이드

PART 1

기본 개념과 설계 원칙

워크플로 vs 에이전트 · 시스템 3계층 · '단순하게 시작' 원칙

PART 2

하네스 엔지니어링

모델을 감싸는 실행 계층의 구조와 설계 체크리스트

PART 3

컨텍스트 엔지니어링

유한 자원으로서의 컨텍스트와 4가지 핵심 전략

PART 4

도구·표준·멀티 에이전트

도구/스킬 설계 · MCP·A2A·AGENTS.md · 멀티 에이전트 확장 시점

PART 5

평가와 신뢰성

자체 평가 파이프라인 · 가드레일과 HITL · 벤치마크 주의점

워크플로 vs 에이전트 — 무엇을 만들 것인가

두 가지 에이전틱 시스템

워크플로

LLM과 도구를 사전 정의된 코드 경로로 오케스트레이션 — 예측 가능성·일관성·저비용

에이전트

LLM이 스스로 프로세스·도구 사용을 동적으로 지휘 — 유연성을 얻고 지연·비용을 지불

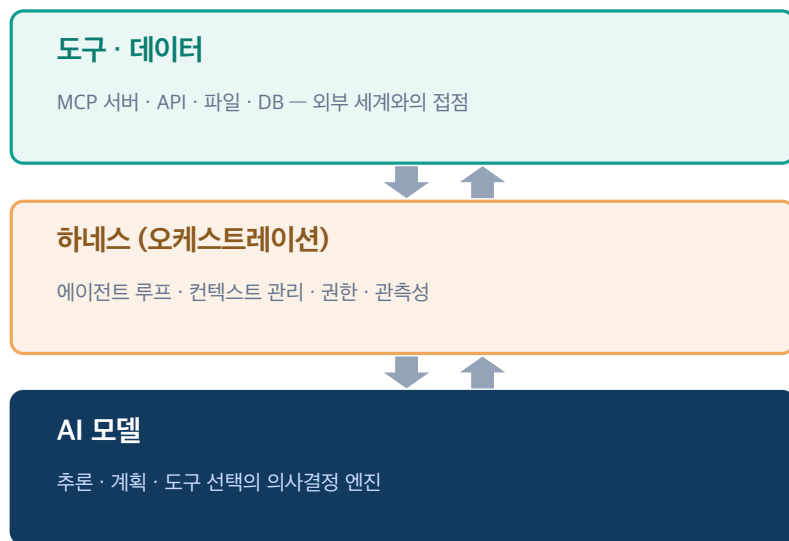
어느 쪽을 쓸까

- 단일 LLM 호출로 충분하면
에이전트 불필요 — 요약·분류·번역은 검색·예시 보강으로 해결
- 예측 가능·구조화된 작업이면
워크플로 — 정해진 단계, 낮은 지연·비용
- 개방형·동적·단계 예측 불가면
에이전트 — 자율 루프, 단 통제 장치 필수

“가장 성공적인 구현은 복잡한 프레임워크가 아니라 단순하고 조합 가능한 패턴을 쓴다” — Anthropic

에이전트 시스템의 3계층

구조



모델은 추론하고, 하네스는 통제하고, 도구는 실행한다

모델

에이전트의 두뇌. 단, 모델 단독으로 에이전트가 아니다 — 요청 해석, 단계 계획, 도구 선택이라는 의사결정만 담당한다.

하네스

신뢰성의 80%가 결정되는 곳. 모델이 제안한 행동을 검증·인가·실행·기록한다.
→ PART 2에서 상세히.

도구·데이터

에이전트의 손발. 2026년엔 직접 통합 대신 MCP 표준으로 연결하는 것이 기본 값. → PART 4에서 상세히.

설계 원칙 — 단순하게 시작하라

1

단일 에이전트부터

모델+도구+프롬프트의 단일 루프로 시작해 핵심 로직·도구 정의를 먼저 다듬는다. 멀티는 그 다음.

2

복잡도는 측정 후에

패턴 추가는 성능 측정이 정당화할 때만. 에이전틱 시스템은 성능을 위해 지연·비용을 지불하는 구조다.

3

추상화에 숨지 말 것

프레임워크를 쓰더라도 내부에서 모델이 무엇을 보고 무엇을 하는지(프롬프트·루프)를 이해하고 시작한다.

확장이 필요할 때 — 먼저 검토할 5가지 워크플로 패턴 (Anthropic)

프롬프트 체이닝

단계 분해, 출력→입력

라우팅

입력 분류 후 분기

병렬화

동시 실행·투표

오케스트레이터-워커

동적 분해·위임

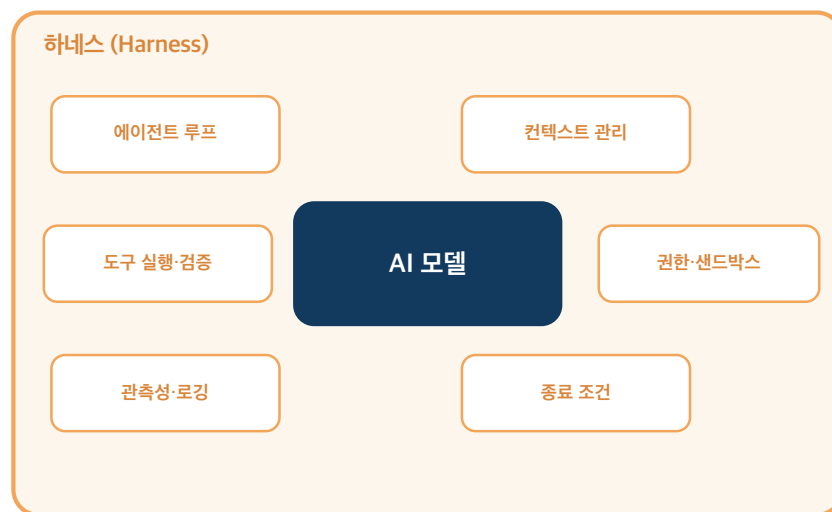
평가자-최적화자

생성↔평가 반복

→ 이 패턴들의 멀티 에이전트 버전은 이전 발표 '에이전틱 AI 시스템 설계 패턴 선택' 참고

하네스 — 모델을 감싸는 실행 계층

구조



모델이 제안한 모든 행동을 검증·인가·실행·기록하는 결정론적 계층

정의

하네스 엔지니어링 = '모델이 아닌 모든 것'(환경·도구·제약·피드백 루프)을 설계하는 분야. 2026년 초 업계 공통 용어로 정착했다.

핵심 통찰

OpenAI·Anthropic·LangChain의 필드 리포트가 수렴한 결론 — 에이전트 신뢰성은 모델보다 모델을 감싼 시스템에 좌우된다.

없으면 생기는 일

토큰 누수, 통제 불능 루프, 위험한 명령 실행. 하네스가 있어야 예측 가능성·보안·관측성을 얻는다.

하네스 설계 체크리스트

도구 실행 · 검증

모델이 제안한 액션을 스키마로 검증한 뒤 실행. 실패는 예외가 아니라 구조화된 에러 메시지로 모델에 돌려줘 스스로 수정하게 한다.

권한 · 샌드박스

최소 권한이 기본값. 파일·네트워크는 격리된 환경에서, 위험 작업(쓰기·결제·삭제)은 허용목록과 승인 게이트로 통제한다.

관측성 · 로깅

모든 도구 호출·결정·중간 추론을 트레이스로 남긴다. 토큰·비용·지연 모니터링 까지 — 디버깅과 평가의 기반이 된다.

루프 제어 · 종료 조건

최대 반복 횟수, 타임아웃, 비용 예산 등 명시적 종료 조건은 필수. 설계 패턴 발효의 '루프 패턴' 무한 루프 경고와 같은 원리다.

Tip — Claude Agent SDK·OpenAI Agents SDK가 위 요소 대부분을 기본 제공: 바닥부터 만들기 전에 SDK 하네스에서 출발할 것

컨텍스트는 유한한 자원이다

왜 '컨텍스트 엔지니어링'인가

프롬프트 엔지니어링의 다음 단계 — 질문이 “어떤 단어를 쓸까”에서 “**어떤 컨텍스트 구성이 원하는 행동을 끌어내는가**”로 바뀐다.

- **Context rot**
컨텍스트가 길어질수록 회수 정확도가 떨어진다는 창이 커져도 성능이 따라오지 않음
- **조용한 킬러**
불필요·비구조 정보의 유입이 에이전트 신뢰성을 가장 흔하게 무너뜨린다
- **목표**
“원하는 결과를 내는 최소한의 고신호(high-signal) 토큰 집합”을 유지

에이전트 컨텍스트의 구성

시스템 프롬프트

‘적절한 고도’로 — 하드코딩된 규칙 나열과 모호한 지시 사이의 균형

도구 정의

도구 설명·인자 스키마도 토큰 — 겹치지 않는 소수 정예로

예시 (Few-shot)

규칙 나열 대신 대표적·다양한 정범례 몇 개가 더 효율적

메시지 히스토리·검색 데이터

가장 빨리 부푸는 영역 — 다음 장의 4가지 전략으로 관리

컨텍스트 관리 — 4가지 핵심 전략

1 · 큐레이션

최소 고신호 토큰

시스템 프롬프트는 명확한 구조로, 불필요 정보는 제거. '더 넣을 게 없을 때'가 아니라 '뺄 게 없을 때' 완성된다.

2 · Just-in-Time 검색

사전 적재 대신 필요 시점 로드

모든 데이터를 미리 넣지 말고 파일 경로·쿼리·링크 같은 가벼운 식별자만 유지, 필요할 때 도구로 로드. Agent Skills의 점진적 공개와 같은 원리.

3 · 컴팩션 · 노트

장기 작업의 지속

창이 차면 대화를 요약·압축(컴팩션)하고, 핵심 상태는 컨텍스트 밖 구조화된 노트(메모리 파일)에 기록해 이어간다.

4 · 서브에이전트 격리

탐색과 종합의 분리

대량 탐색은 서브에이전트의 별도 컨텍스트에서 수행하고, 본 에이전트에는 정제된 요약만 반환 — 멀티 에이전트 패턴의 컨텍스트 버전.

도구와 스킬 설계

좋은 도구의 조건

- **적게, 겹치지 않게**
비슷한 도구가 많으면 오선택 증가 — 목적이 분명한 소수 정예로 구성
- **도구 명세가 곧 UX**
이름·설명·인자 스키마·에러 메시지를 '에이전트가 읽는 문서'로 설계
- **반환은 간결·구조화**
장황한 응답은 컨텍스트를 오염 — 필요한 필드만, 토큰 효율적으로
- **실패를 가르쳐라**
에러 메시지에 원인과 다음 행동 힌트를 담아 자가 수정 유도

Agent Skills — 역량의 패키징

폴더 + SKILL.md(지침·스크립트·참고자료)로 조직의 노하우를 에이전트 역량으로 패키징하는 방식 (Anthropic, 2025.10).

점진적 공개

평소엔 이름·설명만 노출, 작업과 맞을 때만 본문 로드 — 컨텍스트 절약

이식성

같은 스킬을 Claude·SDK·앱 어디서든 재사용

조직 지식의 코드화

브랜드 가이드·보고 양식·사내 절차를 스킬로 배포

표준으로 연결하라 — MCP · A2A · AGENTS.md

2025.12 Agentic AI Foundation(AAIF) 출범 — Anthropic·OpenAI·Block 공동 설립(Linux Foundation 산하). 커스텀 통합을 만들기 전에 아래 표준부터 검토

MCP

도구·데이터 연결

에이전트 ↔ 도구·데이터의 개방 표준. 10,000+ 서버 생태계, 전 업계 채택 — 도구 통합의 기본값.

이럴 때: 내부 시스템·외부 SaaS를 에이전트에 연결할 때

A2A

에이전트 간 협업

서로 다른 벤더·프레임워크 에이전트의 발견·인증·협업 프로토콜(v1.2, 150+ 조직). 결제 확장 AP2.

이럴 때: 팀·조직 경계를 넘는 멀티 에이전트를 설계할 때

AGENTS.md

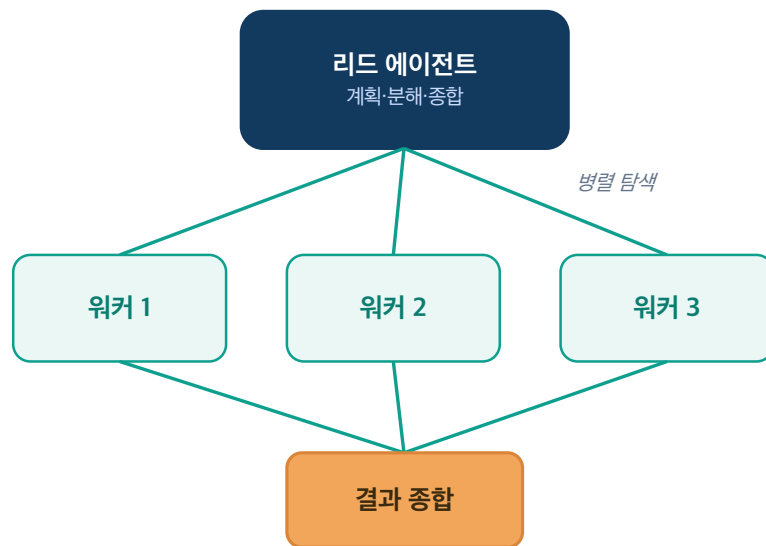
작업 지침 표준

코드베이스 작업 규칙을 저장소의 표준 파일 하나로 — 어떤 코딩 에이전트든 공통 인식.

이럴 때: 코딩 에이전트가 우리 저장소에서 일하게 할 때

멀티 에이전트로 확장하는 시점

오케스트레이터-워커



언제 확장하나

도구가 많아지고 책임이 늘며 단일 에이전트의 성능이 떨어질 때(오선택·지연·실패). 그 전엔 ReAct 루프 개선이 먼저다.

실전 근거

Anthropic 리서치 시스템: 멀티(Opus 4 리드+Sonnet 4 워커)가 단일 대비 내부 평가 +90.2%, 병렬화로 조사 시간 최대 90% 단축.

대가

토큰 사용량은 일반 채팅의 ~15배 — 토큰 비용을 상회하는 가치의 작업에만. 디버깅·평가 복잡도도 함께 증가한다.

평가와 신뢰성 — 검증 없이 프로덕션 없다

자체 평가 파이프라인

작은 샘플로 즉시 시작 — 트레이스 단위 채점, LLM 심판 + 인간 검수 병행. AgentKit Evals·LangSmith 등이 트레이스 채점을 내장.

가드레일 + 사람 개입

가드레일은 계층화(관련성·안전·PII)하고, 고위험 액션(결제·삭제·발송)은 승인 게이트로. 설계 패턴 발표의 HITL 패턴이 여기 해당.

벤치마크는 참고만

τ^2 -bench(정책 준수)·Terminal-Bench(CLI)·OSWorld(컴퓨터 사용)로 위치 확인은 하되 — 2026.04 버클리 RDI가 주요 벤치마크 8종의 리워드 해킹을 입증. 맹신 금지.

평가 항목은 정확도만이 아니다 — 정책 준수·안전·비용·지연을 함께 측정할 것

KEY TAKEAWAYS

정리 — 기억할 4가지

1

단순하게 시작, 측정하며 확장

단일 에이전트+워크플로 패턴부터. 멀티 에이전트는 +90.2% 같은 근거가 ~15배 비용을 정당화할 때

2

신뢰성은 하네스에서 나온다

검증·권한·관측성·종료 조건 — 모델이 아닌 '감싼 시스템'을 설계하라. SDK 하네스가 출발점

3

컨텍스트는 유한 자원

큐레이션·JIT 검색·컴팩션·서브에이전트 격리 — 최소 고신호 토큰을 유지하는 팀이 이긴다

4

표준 위에 구축, 자체 평가로 검증

연결은 MCP·A2A·AGENTS.md로, 검증은 벤치마크가 아닌 자체 평가 파이프라인으로

REFERENCES

참고 자료

- **Building Effective Agents** Anthropic · 2024.12
anthropic.com/research/building-effective-agents
- **Building agents with the Claude Agent SDK** Anthropic · 2025.09
anthropic.com/engineering/building-agents-with-the-claude-agent-sdk
- **Effective context engineering for AI agents** Anthropic · 2025.09
anthropic.com/engineering/effective-context-engineering-for-ai-agents
- **Equipping agents with Agent Skills** Anthropic · 2025.10
anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills
- **How we built our multi-agent research system** Anthropic · 2025.06
anthropic.com/engineering/multi-agent-research-system
- **A Practical Guide to Building Agents** OpenAI · 2025
openai.com/business/guides-and-resources
- **Introducing AgentKit · Agents SDK** OpenAI · 2025.10 / 2026.04
openai.com/index/introducing-agentkit
- **Agent Framework 1.0** Microsoft · 2026.04
learn.microsoft.com/agent-framework
- **Agents 백서 · 에이전틱 AI 설계 패턴 선택** Google · Kaggle / Cloud(2026.05 갱신)
docs.cloud.google.com/architecture
- **MCP · A2A · AGENTS.md (AAIF)** Linux Foundation · 2025.12 출범
modelcontextprotocol.io · a2a-protocol.org · agents.md
- **LangGraph 1.0** LangChain · 2025.10
blog.langchain.com/langchain-langgraph-1dot0
- **ReAct · Reflexion** Yao 2022 · Shinn 2023
arxiv.org/abs/2210.03629 · [2303.11366](https://arxiv.org/abs/2303.11366)
- **τ²-bench · Terminal-Bench · OSWorld · GAIA** 에이전트 평가 벤치마크
arxiv.org/abs/2406.12045 외
- **Agent Harness Engineering** Addy Osmani 외 · 2026
addyosmani.com/blog/agent-harness-engineering

모든 항목 2026-06 웹 검색으로 검증 · 본문 수치(+90.2%, ~15배, 10,000+ 서버 등)는 각 1차 출처 기준